

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

**Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей**

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 12

Дисциплина: Архитектура компьютера

«Программирование в командном процессоре ОС UNIX»

Студент: Пириев Керем

Группа: НПИбд 02–25

Москва, 2026 г.

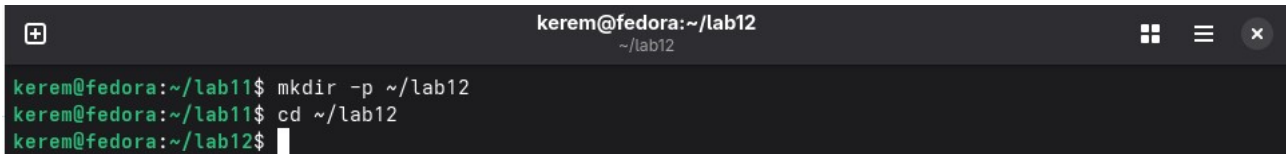
Цель работы

Изучить основы программирования в оболочке `bash`: переменные, циклы, условия, работа с аргументами, тестирование файлов.

1. Создание рабочего каталога

Создан каталог `~/lab12` для размещения всех скриптов:

```
mkdir -p ~/lab12
cd ~/lab12
```

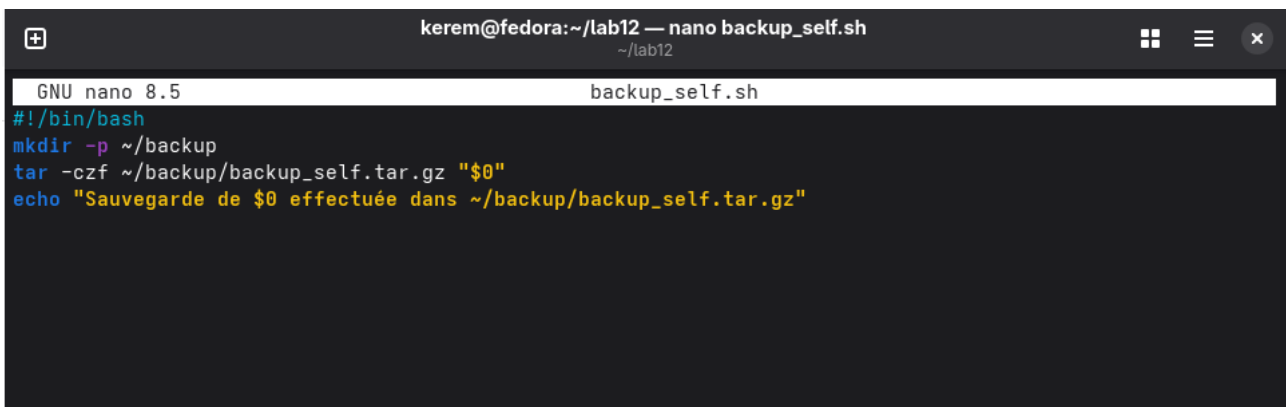
A terminal window titled 'kerem@fedora:~/lab12' with a dark background. It shows the execution of 'mkdir -p ~/lab12' and 'cd ~/lab12' commands. The prompt changes from '~/' to '~/lab12' after the second command.

```
kerem@fedora:~/lab12$ mkdir -p ~/lab12
kerem@fedora:~/lab12$ cd ~/lab12
kerem@fedora:~/lab12$
```

2. Скрипт 1: резервное копирование самого себя

Создан файл `backup_self.sh` с помощью `nano`:

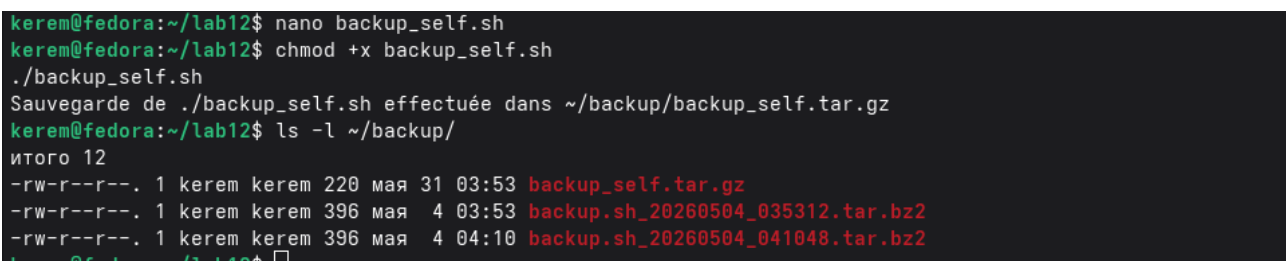
```
nano backup_self.sh
```

A terminal window titled 'kerem@fedora:~/lab12 — nano backup_self.sh' showing the nano editor interface. The script content is visible: shebang, mkdir, tar command, and an echo statement.

```
GNU nano 8.5 backup_self.sh
#!/bin/bash
mkdir -p ~/backup
tar -czf ~/backup/backup_self.tar.gz "$0"
echo "Sauvegarde de $0 effectuée dans ~/backup/backup_self.tar.gz"
```

Запуск и результат:

```
chmod +x backup_self.sh
./backup_self.sh
```

A terminal window showing the execution of the script. It includes the commands to run nano, set permissions, and execute the script. The output shows the backup being performed and a subsequent 'ls -l' command listing the files in the backup directory.

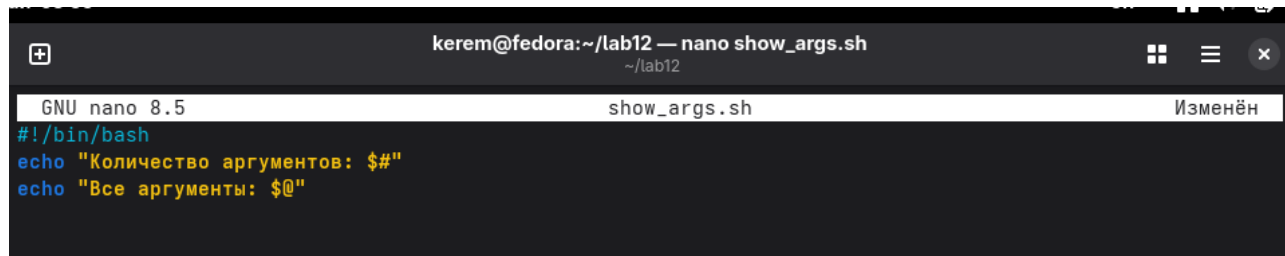
```
kerem@fedora:~/lab12$ nano backup_self.sh
kerem@fedora:~/lab12$ chmod +x backup_self.sh
kerem@fedora:~/lab12$ ./backup_self.sh
Sauvegarde de ./backup_self.sh effectuée dans ~/backup/backup_self.tar.gz
kerem@fedora:~/lab12$ ls -l ~/backup/
итого 12
-rw-r--r--. 1 kerem kerem 220 мая 31 03:53 backup_self.tar.gz
-rw-r--r--. 1 kerem kerem 396 мая  4 03:53 backup.sh_20260504_035312.tar.bz2
-rw-r--r--. 1 kerem kerem 396 мая  4 04:10 backup.sh_20260504_041048.tar.bz2
kerem@fedora:~/lab12$
```

3. Скрипт 2: обработка произвольного числа аргументов

Создан файл `show_args.sh`:

```
nano show_args.sh
```

Содержимое `show_args.sh` (скрипт, который выводит все переданные ему аргументы):

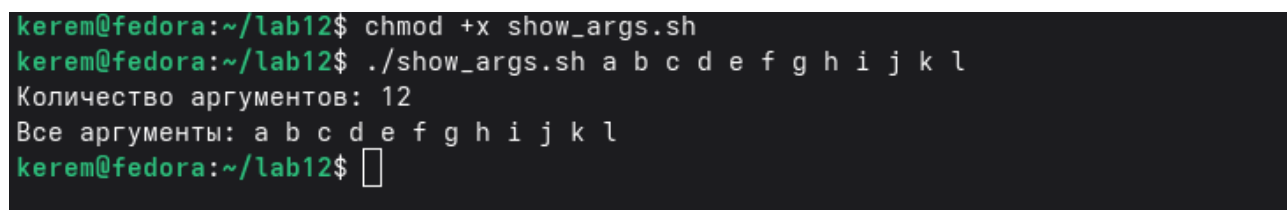
A screenshot of a terminal window with the nano text editor open. The title bar shows 'kerem@fedora: ~/lab12 — nano show_args.sh'. The editor shows the following code:

```
GNU nano 8.5 show_args.sh Изменён
#!/bin/bash
echo "Количество аргументов: $#"
```

Запуск с 12 аргументами:

```
chmod +x show_args.sh
```

```
./show_args.sh a b c d e f g h i j k l
```

A screenshot of a terminal window showing the execution of the script. The commands and their outputs are:

```
kerem@fedora:~/lab12$ chmod +x show_args.sh
kerem@fedora:~/lab12$ ./show_args.sh a b c d e f g h i j k l
Количество аргументов: 12
Все аргументы: a b c d e f g h i j k l
kerem@fedora:~/lab12$
```

4. Скрипт 3: аналог команды `ls` (без использования `ls`)

Создан файл `my_ls.sh`:

```
nano my_ls.sh
```

Содержимое `my_ls.sh` (скрипт выводит содержимое каталога с правами доступа, эмулируя `ls -l`):

A screenshot of a terminal window with the nano text editor open. The title bar shows 'kerem@fedora: ~/lab12 — nano my_ls.sh'. The editor shows the following code:

```
GNU nano 8.5 my_ls.sh Изменён
#!/bin/bash
dir="${1:-.}"
for file in "$dir"/*; do
    if [ -e "$file" ]; then
        permissions=$(ls -ld "$file" | cut -d' ' -f1)
        echo "$permissions $(basename "$file")"
    fi
done
```

Запуск для текущего каталога и `/etc`:

```
chmod +x my_ls.sh
```

```
./my_ls.sh
```

```
./my_ls.sh /etc
```

```

kerem@fedora:~/lab12$ chmod +x my_ls.sh
kerem@fedora:~/lab12$ ./my_ls.sh
-rwxr-xr-x. backup_self.sh
-rwxr-xr-x. my_ls.sh
-rwxr-xr-x. show_args.sh
kerem@fedora:~/lab12$ ./my_ls.sh /etc
drwxr-xr-x. abrt
-rw-r--r--. adjtime
-rw-r--r--. aliases
drwxr-xr-x. alsa
drwxr-xr-x. alternatives
drwxr-xr-x. anaconda
-rw-r--r--. anacrontab
-rw-r--r--. anthy-unicode.conf
-rw-r--r--. asound.conf

```

5. Скрипт 4: подсчёт файлов по расширению

Создан файл `count_ext.sh`:

```
nano count_ext.sh
```

Содержимое `count_ext.sh` (после исправления):

```

kerem@fedora:~/lab12 — nano count_ext.sh
~ /lab12
GNU nano 8.5 count_ext.sh Изменён
#!/bin/bash
ext="$1"
dir="${2:-.}"
count=$(find "$dir" -type f -name "$*$ext" 2>/dev/null | wc -l)
echo "Nombre de fichiers avec l'extension $ext dans $dir : $count"

```

Запуск после исправления:

```

chmod +x count_ext.sh
./count_ext.sh .txt /etc

```

```

kerem@fedora:~/lab12$ nano count_ext.sh
kerem@fedora:~/lab12$ chmod +x count_ext.sh
kerem@fedora:~/lab12$ ./count_ext.sh .txt /etc
Nombre de fichiers avec l'extension .txt dans /etc : 20
kerem@fedora:~/lab12$

```

Выводы

В ходе работы были написаны и отлажены четыре bash-скрипта:

1. **Резервное копирование самого себя** – скрипт создаёт архив с собственным исходным кодом.
2. **Обработка произвольного количества аргументов** – скрипт выводит число и список всех переданных аргументов.
3. **Эмуляция команды `ls`** – скрипт выводит содержимое каталога с правами доступа без использования встроенной команды `ls`.
4. **Подсчёт файлов заданного расширения** – скрипт подсчитывает количество файлов с указанным расширением в заданном каталоге.

Получены практические навыки работы с переменными, циклами, условиями, передачей параметров и тестированием файлов в оболочке bash.

Ответы на контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (shell) – это программа, обеспечивающая интерфейс между пользователем и операционной системой, интерпретирующая команды, вводимые пользователем.

Примеры: sh (Bourne shell), bash (Bourne again shell), csh, zsh, fish.

Отличия: синтаксис, дополнительные возможности (история команд, автодополнение, поддержка скриптов, настройка приглашения).

2. Что такое POSIX?

POSIX (Portable Operating System Interface) – набор стандартов, описывающих интерфейсы между операционной системой и прикладными программами, обеспечивающий переносимость приложений на уровне исходного кода между различными UNIX-подобными системами.

3. Как определяются переменные и массивы в языке программирования bash?

- **Переменная:** `var=value` (без пробелов вокруг =).
- **Массив:** `arr=(val1 val2 val3)`
- **Доступ к значению переменной:** `$var`
- **Доступ к элементу массива:** `${arr[0]}`

4. Каково назначение операторов `let` и `read`?

- `let` – для выполнения арифметических операций (например, `let "sum = a + b"`).
- `read` – для чтения ввода пользователя (с клавиатуры) и сохранения его в переменные (например, `read name`).

5. Какие арифметические операции можно применять в языке программирования `bash`?

`+` (сложение), `-` (вычитание), `*` (умножение), `/` (деление), `%` (остаток от деления), `+=`, `-=`, `*=`, `/=`, `++` (инкремент), `--` (декремент).

6. Что означает операция `(( ))`?

`(( ))` – конструкция для выполнения арифметических операций в стиле языка C. Позволяет использовать арифметические выражения и операции сравнения без `$` перед переменными (например, `(( a = b + c ))`).

7. Какие стандартные имена переменных Вам известны?

`HOME`, `PATH`, `USER`, `SHELL`, `PWD`, `PS1` (формат приглашения), `IFS` (внутренний разделитель полей), `LOGNAME`.

8. Что такое метасимволы?

Метасимволы – это символы, имеющие специальное значение для командной оболочки: `*`, `?`, `[ ]`, `|`, `&`, `;`, `( )`, `{ }`, `<`, `>`, пробел, табуляция.

9. Как экранировать метасимволы?

Экранирование выполняется с помощью:

- Обратного следа перед метасимволом: `\*`
- Одинарных кавычек: `'*'`
- Двойных кавычек (частичное экранирование): `"*"`

10. Как создавать и запускать командные файлы?

1. Создать текстовый файл с первой строкой `#!/bin/bash`.
2. Сделать файл исполняемым: `chmod +x имя_файла`.
3. Запустить: `./имя_файла` (если файл находится в текущем каталоге) или указать полный путь.

11. Как определяются функции в языке программирования `bash`?

Синтаксис:

`bash`

```
function имя_функции {  
    команды  
}
```

или

bash

```
имя_функции() {  
    команды  
}
```

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

С помощью условных операторов `test` или `[ ]`:

- `test -d файл` – возвращает истину, если файл является каталогом.
- `test -f файл` – возвращает истину, если файл является обычным файлом.

Пример в скрипте:

bash

```
if [ -d "$file" ]; then  
    echo "$file – каталог"  
fi
```

13. Каково назначение команд `set`, `typeset` и `unset`?

- `set` – отображает все переменные окружения и их значения, а также позволяет устанавливать параметры оболочки.
- `typeset` (или `declare`) – объявляет переменные с заданными атрибутами (например, `typeset -i` для целочисленной переменной).
- `unset` – удаляет переменную или функцию.

14. Как передаются параметры в командные файлы?

Параметры передаются через пробел после имени скрипта при запуске:

bash

```
./script.sh param1 param2 param3
```

Внутри скрипта параметры доступны как:

- `$1`, `$2`, ..., `$9` – первый, второй,... девятый параметры;
- `${10}`, `${11}` и т.д. – для параметров, начиная с десятого.

15. Назовите специальные переменные языка `bash` и их назначение.

Переменная	Назначение
<code>\$#</code>	Количество переданных аргументов

Переменная	Назначение
\$*	Все аргументы как одна строка
\$@	Все аргументы как отдельные строки
\$?	Код возврата последней выполненной команды
\$\$	PID (идентификатор процесса) текущей оболочки
\$!	PID последнего фонового процесса
\$0	Имя запущенного скрипта

